Ohjelmistotuotanto

Syksy 2026

Luento 8

17.11.2026

Muistakaa...

Muistakaa...

- Kurssipalaute
  - Kurssilla lopussa kerättävän palautteen lisäksi ns. jatkuva palaute https://norppa.helsinki.fi

Muistakaa...

- Kurssipalaute
  - Kurssilla lopussa kerättävän palautteen lisäksi ns. jatkuva palaute https://norppa.helsinki.fi
- Paja BK107
  - ma 16-19
  - ti 13-16
  - to 16-19
  - pe 13-16
- Kurssin Discord: https://study.cs.helsinki.fi/discord/join/ohtu
  - kanavalle liitytään komennolla */join course TKT20006 - Ohjelmistotuotanto - ohtu*

Muistakaa...

- Kurssipalaute
  - Kurssilla lopussa kerättävän palautteen lisäksi ns. jatkuva palaute https://norppa.helsinki.fi
- Paja BK107
  - ma 16-19
  - ti 13-16
  - to 16-19
  - pe 13-16
- Kurssin Discord: https://study.cs.helsinki.fi/discord/join/ohtu
  - kanavalle liitytään komennolla */join course TKT20006 - Ohjelmistotuotanto - ohtu*
- Miniprojekti

Muistakaa...

- Kurssipalaute
  - Kurssilla lopussa kerättävän palautteen lisäksi ns. jatkuva palaute https://norppa.helsinki.fi
- Paja BK107
  - ma 16-19
  - ti 13-16
  - to 16-19
  - pe 13-16
- Kurssin Discord: https://study.cs.helsinki.fi/discord/join/ohtu
  - kanavalle liitytään komennolla */join course TKT20006 - Ohjelmistotuotanto - ohtu*
- Miniprojekti
- Kurssikoe tiistaina 15.12. klo 13-16 A111 ja B123
  - **ilmoittaudu viimeistään la 5.12. klo 23.59!**

Ohjelmiston elinkaaren vaiheet

- Riippumatta tyylistä ja tavasta, jolla ohjelmisto tehdään, ohjelmistojen tekemiseen kuuluu
  - vaatimusten analysointi ja määrittely
  - **suunnittelu**
  - **toteutus**
  - testaus/laadunhallinta
  - ohjelmiston ylläpito

Ohjelmiston elinkaaren vaiheet

- Riippumatta tyylistä ja tavasta, jolla ohjelmisto tehdään, ohjelmistojen tekemiseen kuuluu
  - vaatimusten analysointi ja määrittely
  - **suunnittelu**
  - **toteutus**
  - testaus/laadunhallinta
  - ohjelmiston ylläpito
- Jakautuu kahteen vaiheeseen:
  - arkkitehtuurisuunnittelu
  - olio/komponenttisuunnittelu

Ohjelmiston elinkaaren vaiheet

- Riippumatta tyylistä ja tavasta, jolla ohjelmisto tehdään, ohjelmistojen tekemiseen kuuluu
  - vaatimusten analysointi ja määrittely
  - **suunnittelu**
  - **toteutus**
  - testaus/laadunhallinta
  - ohjelmiston ylläpito
- Jakautuu kahteen vaiheeseen:
  - arkkitehtuurisuunnittelu
  - olio/komponenttisuunnittelu
- Näiden lisäksi UI/UX-suunnittelu

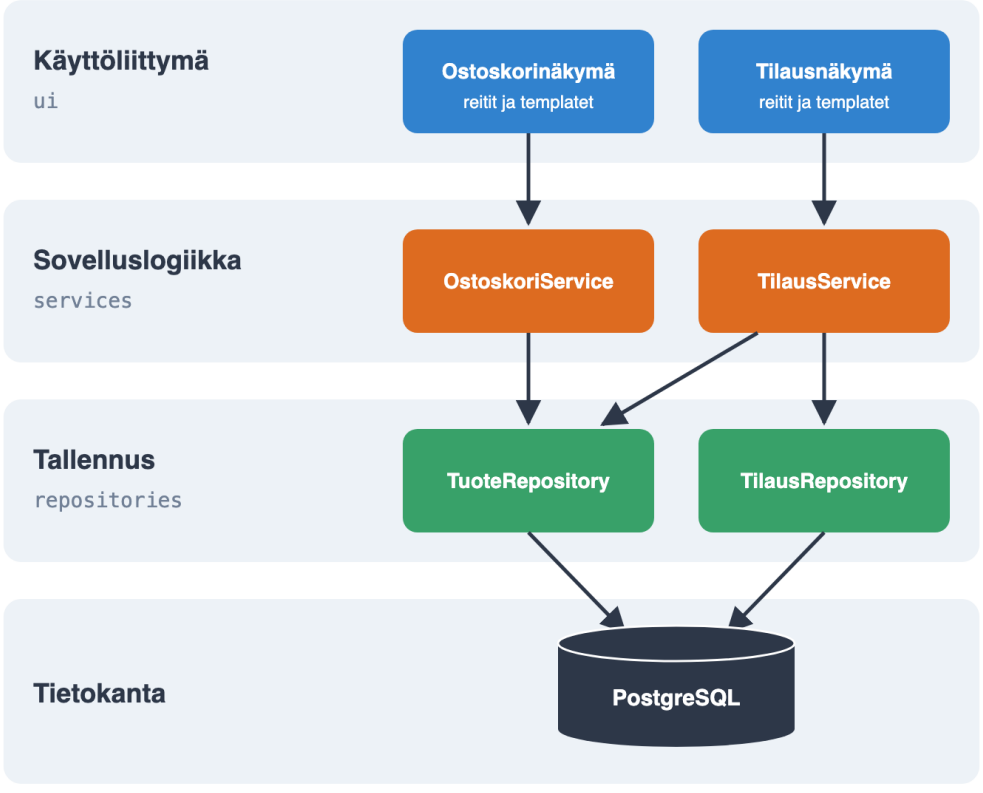
Arkkitehtuurityyli

Arkkitehtuurityyli

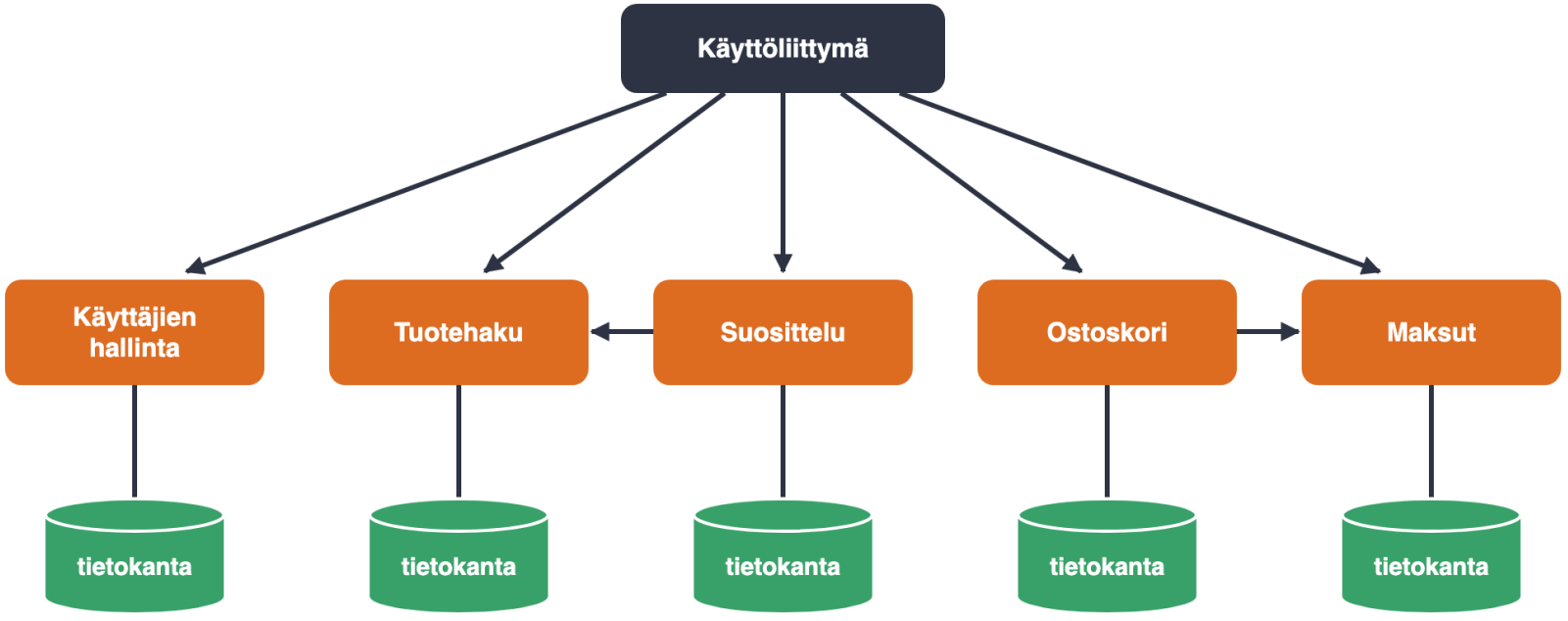
- Ohjelmiston arkkitehtuuri perustuu yleensä yhteen tai useampaan **arkkitehtuurityyliin** (architectural style)
  - hyväksi havaittu tapa strukturoida tietyntyyppisiä sovelluksia
- Tyylejä suuri määrä
  - Kerrosarkkitehtuuri
  - Mikropalveluarkkitehtuuri
  - MVC
  - Pipes-and-filters
  - Palveluperustainen arkkitehtuuri
  - Client-server
  - Publish-subscribe
  - Event driven
  - REST
  - ...

Kerrosarkkitehtuuri

- *Kerros* on kokoelma toisiinsa liittyviä olioita, jotka muodostavat toiminnallisuuden suhteen loogisen kokonaisuuden



Mikropalveluarkkitehtuuri



kaikki kommunikointi verkon yli, esim. REST · jokaisella palvelulla oma tietokanta

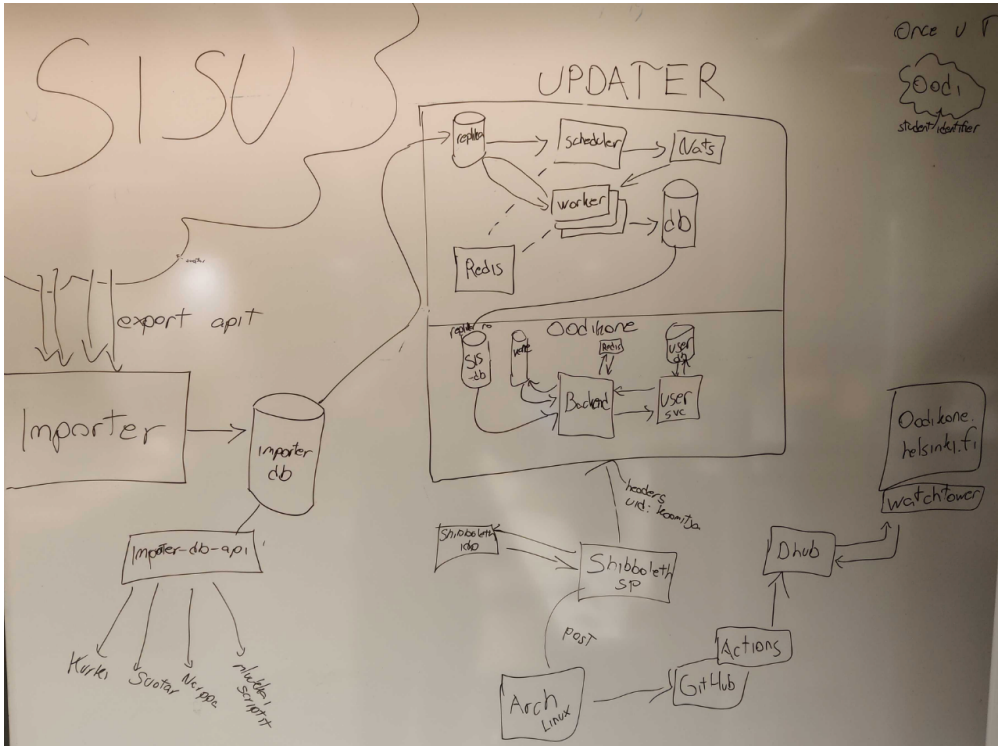
- sovellus kootaan useista (jopa sadoista) pienistä verkossa toimivista autonomisista palveluista

Arkkitehtuurin kuvaamisesta

- Ei yhtä standardoitua notaatiota, useimmiten epäformaaleja laatikko/nuoli-kaavioita
  - UML:n komponentti- ja pakkauskaaviot olivat suosittuja 2000-luvulla, nykyään harvinaisia

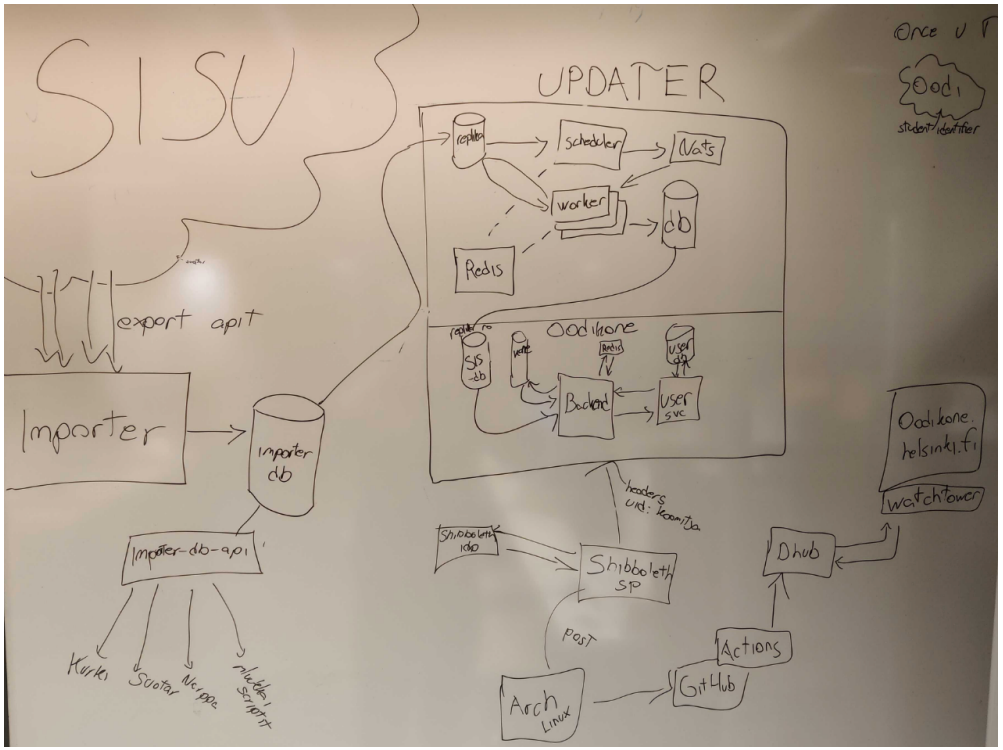
Arkkitehtuurin kuvaamisesta

- Ei yhtä standardoitua notaatiota, useimmiten epäformaaleja laatikko/nuoli-kaavioita
  - UML:n komponentti- ja pakkauskaaviot olivat suosittuja 2000-luvulla, nykyään harvinaisia



Arkkitehtuurin kuvaamisesta

- Ei yhtä standardoitua notaatiota, useimmiten epäformaaleja laatikko/nuoli-kaavioita
  - UML:n komponentti- ja pakkauskaaviot olivat suosittuja 2000-luvulla, nykyään harvinaisia



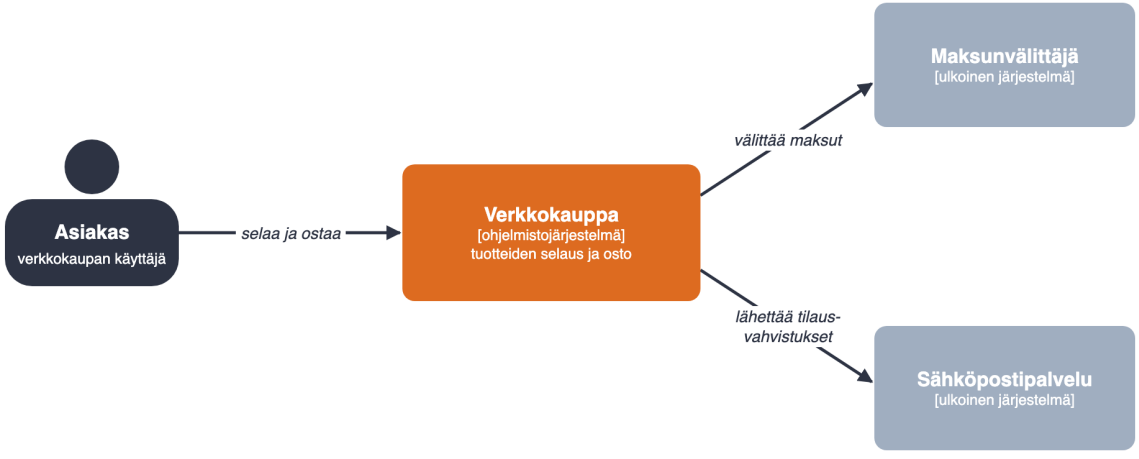
- Kaavioita yhä useammin koodina (esim. Mermaid), versioidaan koodin rinnalla

C4-malli

- Suosiota saavuttanut *C4-malli*: neljä tarkentuvaa tasoa
  - konteksti (system context), containerit (containers), komponentit (components), koodi (code)
  - jokaisella kaaviolla selkeä taso, jokainen laatikko ja nuoli nimetty

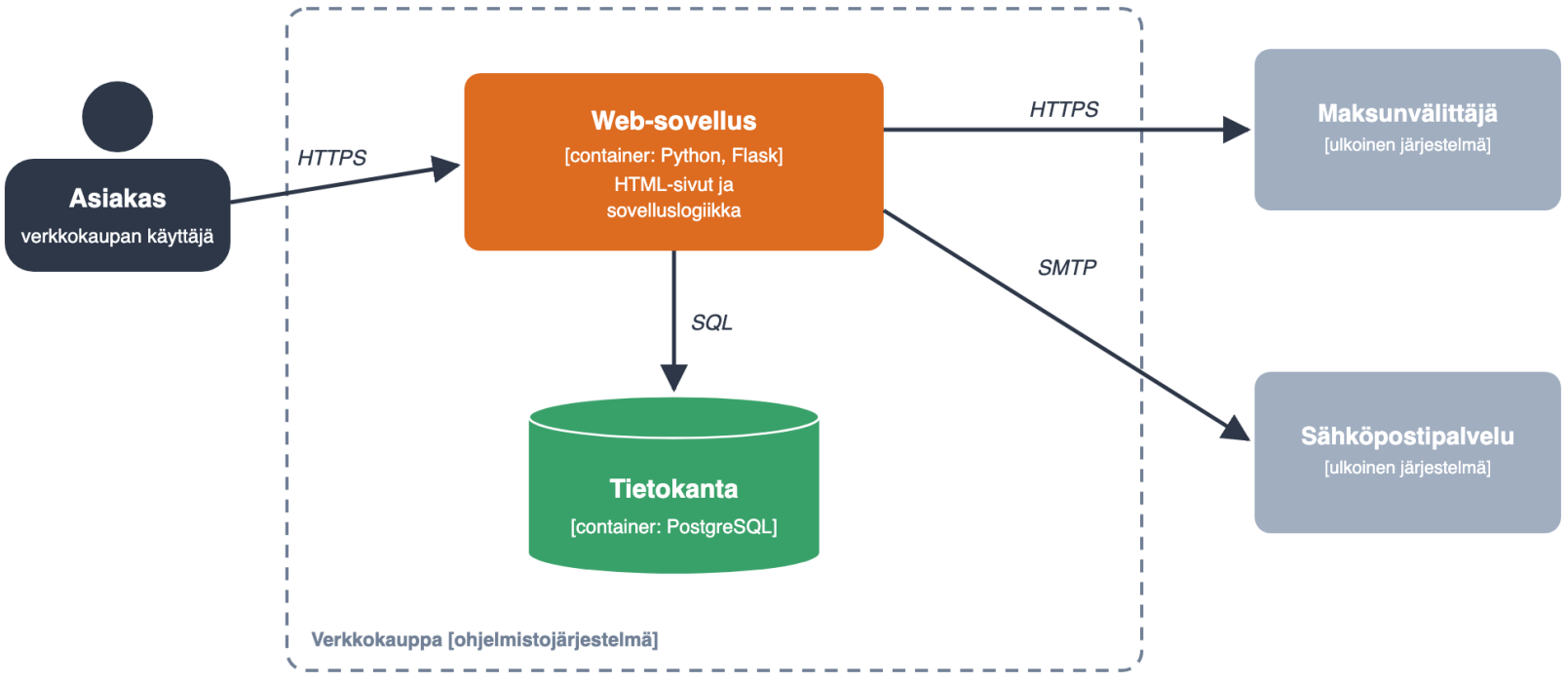
C4-malli

- Suosiota saavuttanut *C4-malli*: neljä tarkentuvaa tasoa
  - konteksti (system context), containerit (containers), komponentit (components), koodi (code)
  - jokaisella kaaviolla selkeä taso, jokainen laatikko ja nuoli nimetty



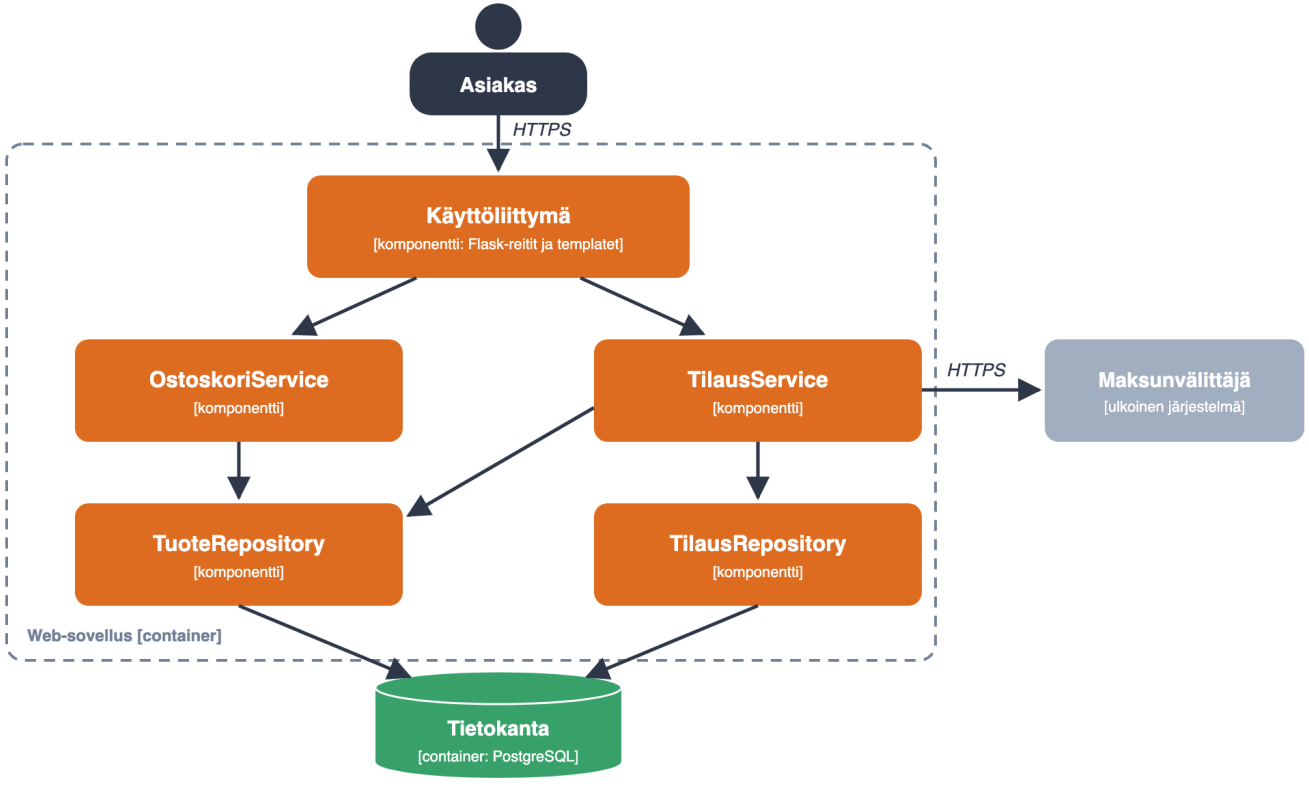
- Konteksti (system context): järjestelmä, sen käyttäjät ja ulkoiset järjestelmät

C4-malli: containerit



- Containerit (containers): erikseen ajettavat osat, kuten sovellukset ja tietokannat, sekä niiden kommunikointi

C4-malli: komponentit



- Komponentit (components): yhden containerin (web-sovellus) sisäinen rakenne

Arkkitehtuuripäätösten kirjaaminen

- Ei pelkkiä kuvia: hyödyllinen arkkitehtuurikuvaus dokumentoi ja perustelee tehtyjä *arkkitehtuurisia valintoja*

Arkkitehtuuripäätösten kirjaaminen

- Ei pelkkiä kuvia: hyödyllinen arkkitehtuurikuvaus dokumentoi ja perustelee tehtyjä *arkkitehtuurisia valintoja*
- *Architecture Decision Record* (ADR)
  - lyhyt Markdown-tiedosto jokaisesta merkittävästä päätöksestä, versionhallintaan koodin rinnalle
  - konteksti, päätös ja seuraukset
  - vanhaa ei muokata, muuttunut päätös kirjataan uutena ADR:nä

ADR-esimerkki

```
# ADR 3: Tietokannaksi PostgreSQL

Tila: hyväksytty 12.3.2026

## Konteksti
Tilaus- ja tuotedata on relaatioista ja
vaatii transaktioita. Tiimi osaa SQL:n.

## Päätös
Käytetään pilvipalvelun hallinnoimaa
PostgreSQL:ää.

## Seuraukset
+ tilauksille transaktiot ja eheys
+ ei omaa tietokantapalvelimen ylläpitoa
- skeemamuutokset vaativat migraatiot
- sidos pilvipalveluntarjoajaan
```

Arkkitehtuuri ketterissä menetelmissä

Arkkitehtuuri ketterissä menetelmissä

- Ketterien menetelmien kantava teema on toimivan, asiakkaalle arvoa tuottavan ohjelmiston nopea toimittaminen

Arkkitehtuuri ketterissä menetelmissä

- Ketterien menetelmien kantava teema on toimivan, asiakkaalle arvoa tuottavan ohjelmiston nopea toimittaminen
- Arkkitehtuurin suunnittelu ja dokumentointi on perinteisesti pitkäkestoinen, ohjelmoinnin aloittamista edeltävä vaihe

Arkkitehtuuri ketterissä menetelmissä

- Ketterien menetelmien kantava teema on toimivan, asiakkaalle arvoa tuottavan ohjelmiston nopea toimittaminen
- Arkkitehtuurin suunnittelu ja dokumentointi on perinteisesti pitkäkestoinen, ohjelmoinnin aloittamista edeltävä vaihe
- Ketterät menetelmät ja “arkkitehtuurivetoinen” ohjelmistotuotanto siis jossain määrin ristiriidassa

Arkkitehtuuri ketterissä menetelmissä

- Ketterien menetelmien kantava teema on toimivan, asiakkaalle arvoa tuottavan ohjelmiston nopea toimittaminen
- Arkkitehtuurin suunnittelu ja dokumentointi on perinteisesti pitkäkestoinen, ohjelmoinnin aloittamista edeltävä vaihe
- Ketterät menetelmät ja “arkkitehtuurivetoinen” ohjelmistotuotanto siis jossain määrin ristiriidassa
- Ketterien menetelmien yhteydessä puhutaan *inkrementaalisesta suunnittelusta ja arkkitehtuurista*

Arkkitehtuuri ketterissä menetelmissä

- Ketterien menetelmien kantava teema on toimivan, asiakkaalle arvoa tuottavan ohjelmiston nopea toimittaminen
- Arkkitehtuurin suunnittelu ja dokumentointi on perinteisesti pitkäkestoinen, ohjelmoinnin aloittamista edeltävä vaihe
- Ketterät menetelmät ja “arkkitehtuurivetoinen” ohjelmistotuotanto siis jossain määrin ristiriidassa
- Ketterien menetelmien yhteydessä puhutaan *inkrementaalisesta suunnittelusta ja arkkitehtuurista*
- Arkkitehtuuri mietitään riittävällä tasolla projektin alussa
  - Jotkut projektit alkavat ns. nollasprintillä ja alustava arkkitehtuuri määritellään tällöin

Arkkitehtuuri ketterissä menetelmissä

- Ketterien menetelmien kantava teema on toimivan, asiakkaalle arvoa tuottavan ohjelmiston nopea toimittaminen
- Arkkitehtuurin suunnittelu ja dokumentointi on perinteisesti pitkäkestoinen, ohjelmoinnin aloittamista edeltävä vaihe
- Ketterät menetelmät ja “arkkitehtuurivetoinen” ohjelmistotuotanto siis jossain määrin ristiriidassa
- Ketterien menetelmien yhteydessä puhutaan *inkrementaalisesta suunnittelusta ja arkkitehtuurista*
- Arkkitehtuuri mietitään riittävällä tasolla projektin alussa
  - Jotkut projektit alkavat ns. nollasprintillä ja alustava arkkitehtuuri määritellään tällöin
- “lopullinen” arkkitehtuuri muodostuu iteraatio iteraatiolta samalla kun uutta toiminnallisuutta toteutetaan

Inkrementaalinen arkkitehtuuri

- Tehdään alustava suunnitelma arkkitehtuurista

Inkrementaalinen arkkitehtuuri

- Tehdään alustava suunnitelma arkkitehtuurista
- Alussa ns. *walking skeleton*
  - sisältää tynkäversiot ohjelmiston komponenttirakenteesta



Inkrementaalinen arkkitehtuuri

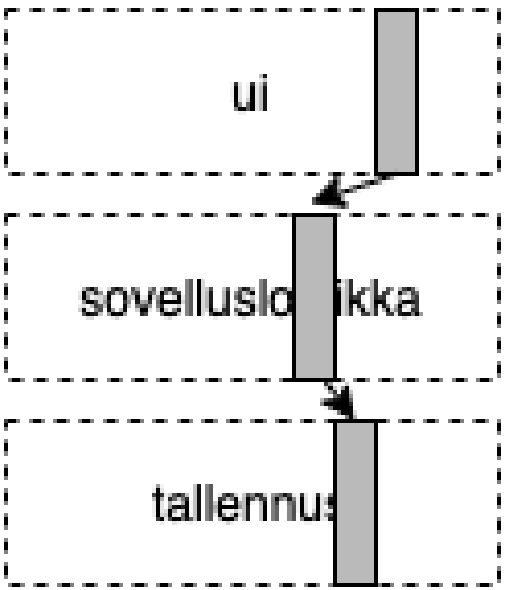
- Tehdään alustava suunnitelma arkkitehtuurista
- Alussa ns. *walking skeleton*
  - sisältää tynkäversiot ohjelmiston komponenttirakenteesta



- Rakennetaan skeletonin varaan tuotetta story storyltä

Inkrementaalinen arkkitehtuuri

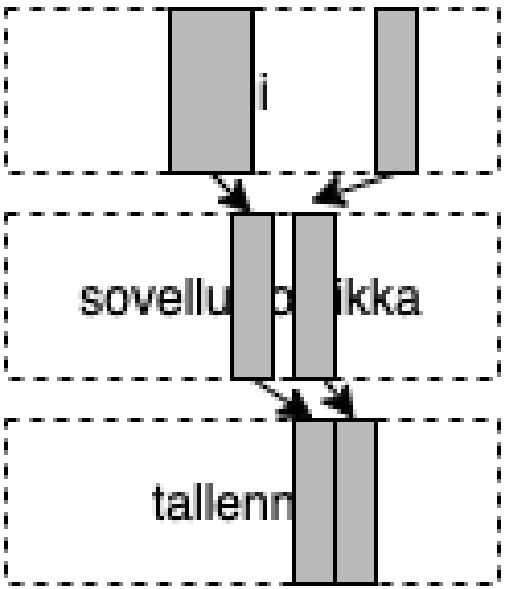
- Tehdään alustava suunnitelma arkkitehtuurista
- Alussa ns. *walking skeleton*
  - sisältää tynkäversiot ohjelmiston komponenttirakenteesta



- Rakennetaan skeletonin varaan tuotetta story storyltä

Inkrementaalinen arkkitehtuuri

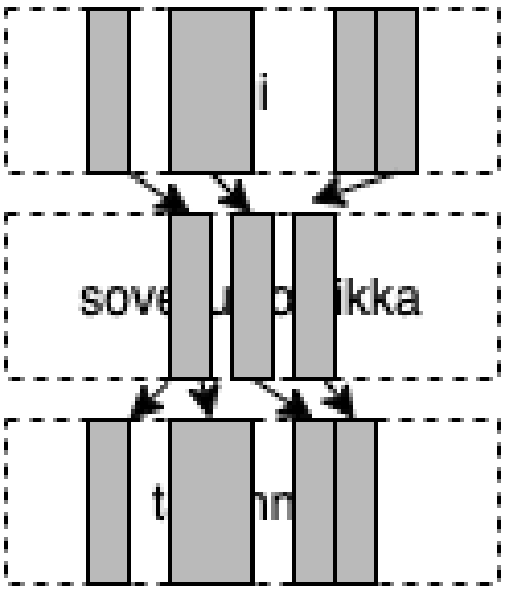
- Tehdään alustava suunnitelma arkkitehtuurista
- Alussa ns. *walking skeleton*
  - sisältää tynkäversiot ohjelmiston komponenttirakenteesta



- Rakennetaan skeletonin varaan tuotetta story storyltä

Inkrementaalinen arkkitehtuuri

- Tehdään alustava suunnitelma arkkitehtuurista
- Alussa ns. *walking skeleton*
  - sisältää tynkäversiot ohjelmiston komponenttirakenteesta



- Rakennetaan skeletonin varaan tuotetta story storyltä

Arkkitehtuuri ketterissä menetelmissä

- Perinteisesti arkkitehtuurin luonut *ohjelmistoarkkitehti*
  - ohjelmoijat velvoitettuja noudattamaan arkkitehtuuria

Arkkitehtuuri ketterissä menetelmissä

- Perinteisesti arkkitehtuurin luonut *ohjelmistoarkkitehti*
  - ohjelmoijat velvoitettuja noudattamaan arkkitehtuuria
- Ketterä ideaali: kehitystiimi luo arkkitehtuurin yhdessä
  - *The best architectures, requirements, and designs emerge from self-organizing teams*

Arkkitehtuuri ketterissä menetelmissä

- Perinteisesti arkkitehtuurin luonut *ohjelmistoarkkitehti*
  - ohjelmoijat velvoitettuja noudattamaan arkkitehtuuria
- Ketterä ideaali: kehitystiimi luo arkkitehtuurin yhdessä
  - *The best architectures, requirements, and designs emerge from self-organizing teams*
- Etuja:
  - kehittäjät sitoutuvat paremmin arkkitehtuurin noudattamiseen kuin “norsunluutornissa” olevan arkkitehdin määrittelemään
  - dokumentaatio voi olla kevyt, tiimi tuntee arkkitehtuurin hengen ja pystyy sitä noudattamaan

Arkkitehtuuri ketterissä menetelmissä

- Perinteisesti arkkitehtuurin luonut *ohjelmistoarkkitehti*
  - ohjelmoijat velvoitettuja noudattamaan arkkitehtuuria
- Ketterä ideaali: kehitystiimi luo arkkitehtuurin yhdessä
  - *The best architectures, requirements, and designs emerge from self-organizing teams*
- Etuja:
  - kehittäjät sitoutuvat paremmin arkkitehtuurin noudattamiseen kuin “norsunluutornissa” olevan arkkitehdin määrittelemään
  - dokumentaatio voi olla kevyt, tiimi tuntee arkkitehtuurin hengen ja pystyy sitä noudattamaan
- Ideaali, ei kaikkialla käytäntö: monissa ketteristäkin organisaatioissa arkkitehteja
  - esim. SAFe:n arkkitehtiroolit, *staff/principal engineer*
  - ketterä arkkitehti tukee tiimejä ja sovittaa ratkaisuja yhteen, ei sanele ylhäältä

Inkrementaalinen arkkitehtuuri: edut ja riskit

- Oletus: optimaalista arkkitehtuuria ei pystytä suunnittelemaan projektin alussa
  - Jo tehtyjä arkkitehtuuriratkaisuja muutetaan tarvittaessa

Inkrementaalinen arkkitehtuuri: edut ja riskit

- Oletus: optimaalista arkkitehtuuria ei pystytä suunnittelemaan projektin alussa
  - Jo tehtyjä arkkitehtuuriratkaisuja muutetaan tarvittaessa
- Arkkitehtuurin suunnittelussa ketterä pyrkii *välttämään liian aikaisin tehtävää, ehkä turhaksi osoittautuvaa työtä*

Inkrementaalinen arkkitehtuuri: edut ja riskit

- Oletus: optimaalista arkkitehtuuria ei pystytä suunnittelemaan projektin alussa
  - Jo tehtyjä arkkitehtuuriratkaisuja muutetaan tarvittaessa
- Arkkitehtuurin suunnittelussa ketterä pyrkii *välttämään liian aikaisin tehtävää, ehkä turhaksi osoittautuvaa työtä*
- Inkrementaalinen arkkitehtuuri edellyttää koodilta hyvää sisäistä laatua ja kehittäjiltä kurinalaisuutta
  - muuten seurauksena *big ball of mud*
  - Martin Fowler: *in its common usage, incremental design is a disaster*

Inkrementaalinen arkkitehtuuri: edut ja riskit

- Oletus: optimaalista arkkitehtuuria ei pystytä suunnittelemaan projektin alussa
  - Jo tehtyjä arkkitehtuuriratkaisuja muutetaan tarvittaessa
- Arkkitehtuurin suunnittelussa ketterä pyrkii *välttämään liian aikaisin tehtävää, ehkä turhaksi osoittautuvaa työtä*
- Inkrementaalinen arkkitehtuuri edellyttää koodilta hyvää sisäistä laatua ja kehittäjiltä kurinalaisuutta
  - muuten seurauksena *big ball of mud*
  - Martin Fowler: *in its common usage, incremental design is a disaster*
- Koodausagentit voimistavat riskiä: ilman ohjausta agentti tekee taktisia ratkaisuja, jotka rapauttavat arkkitehtuuria
  - periaatteet ohjetiedostoon ja ADR:iin, säännöt automaattisesti tarkastettaviksi CI:ssä

Olio/komponenttisuunnittelu

Olio/komponenttisuunnittelu

- Sovelluksen arkkitehtuuri antaa raamit, jotka ohjaavat sovelluksen tarkempaa suunnittelua ja toteuttamista

Olio/komponenttisuunnittelu

- Sovelluksen arkkitehtuuri antaa raamit, jotka ohjaavat sovelluksen tarkempaa suunnittelua ja toteuttamista
- *Olio- tai komponenttisuunnittelu*
  - tarkentaa arkkitehtuuristen komponenttien väliset rajapinnat sekä hahmottelee ohjelman luokka- tai moduulirakenteen

Olio/komponenttisuunnittelu

- Sovelluksen arkkitehtuuri antaa raamit, jotka ohjaavat sovelluksen tarkempaa suunnittelua ja toteuttamista
- *Olio- tai komponenttisuunnittelu*
  - tarkentaa arkkitehtuuristen komponenttien väliset rajapinnat sekä hahmottelee ohjelman luokka- tai moduulirakenteen
- Vesiputousmallissa komponenttisuunnittelu tehty ennen ohjelmointia ja dokumentoitu tarkkaan esim. UML:llä

Olio/komponenttisuunnittelu

- Sovelluksen arkkitehtuuri antaa raamit, jotka ohjaavat sovelluksen tarkempaa suunnittelua ja toteuttamista
- *Olio- tai komponenttisuunnittelu*
  - tarkentaa arkkitehtuuristen komponenttien väliset rajapinnat sekä hahmottelee ohjelman luokka- tai moduulirakenteen
- Vesiputousmallissa komponenttisuunnittelu tehty ennen ohjelmointia ja dokumentoitu tarkkaan esim. UML:llä
- Ketterässä tarkka suunnittelu tehdään yleensä vasta ohjelmoitaessa

Olio/komponenttisuunnittelu

- Sovelluksen arkkitehtuuri antaa raamit, jotka ohjaavat sovelluksen tarkempaa suunnittelua ja toteuttamista
- *Olio- tai komponenttisuunnittelu*
  - tarkentaa arkkitehtuuristen komponenttien väliset rajapinnat sekä hahmottelee ohjelman luokka- tai moduulirakenteen
- Vesiputousmallissa komponenttisuunnittelu tehty ennen ohjelmointia ja dokumentoitu tarkkaan esim. UML:llä
- Ketterässä tarkka suunnittelu tehdään yleensä vasta ohjelmoitaessa
- Suunnittelussa pyritään maksimoimaan *koodin sisäinen laatu*
  - helppo ylläpidettävyys ja laajennettavuus

Olio/komponenttisuunnittelu

- Sovelluksen arkkitehtuuri antaa raamit, jotka ohjaavat sovelluksen tarkempaa suunnittelua ja toteuttamista
- *Olio- tai komponenttisuunnittelu*
  - tarkentaa arkkitehtuuristen komponenttien väliset rajapinnat sekä hahmottelee ohjelman luokka- tai moduulirakenteen
- Vesiputousmallissa komponenttisuunnittelu tehty ennen ohjelmointia ja dokumentoitu tarkkaan esim. UML:llä
- Ketterässä tarkka suunnittelu tehdään yleensä vasta ohjelmoitaessa
- Suunnittelussa pyritään maksimoimaan *koodin sisäinen laatu*
  - helppo ylläpidettävyys ja laajennettavuus
- Ohjelmistosuunnittelu on “enemmän taidetta kuin tiedettä”, kokemus ja hyvien käytänteiden tuntemus auttaa

Laadukas koodi

- Tavoitteena siis **sisäiseltä laadultaan** hyvä koodi

Laadukas koodi

- Tavoitteena siis **sisäiseltä laadultaan** hyvä koodi
- *Sisäinen laatu* (internal quality)
  - onko virheiden jäljitys ja korjaaminen helppoa
  - onko koodia helppo laajentaa ja jatkokehittää
  - pystytäänkö koodin toiminnallisuuden oikeellisuus varmistamaan muutoksia tehtäessä

Laadukas koodi

- Tavoitteena siis **sisäiseltä laadultaan** hyvä koodi
- *Sisäinen laatu* (internal quality)
  - onko virheiden jäljitys ja korjaaminen helppoa
  - onko koodia helppo laajentaa ja jatkokehittää
  - pystytäänkö koodin toiminnallisuuden oikeellisuus varmistamaan muutoksia tehtäessä
- Jos sisäinen laatu rapistuu
  - alkaa vaikuttaa myös ulkoiseen eli käyttäjän kokemaan laatuun
  - kehitystiimin velositeetti alkaa tippua

Laadukkaan koodin tuntomerkkejä

Laadukkaan koodin tuntomerkkejä

- Laadukkaalla koodilla joukko yhteneviä ominaisuuksia, tai *laatuattribuutteja*, esim. seuraavat:
  - kapselointi
  - korkea koheesion aste
  - riippuvuuksien vähäisyys
  - toisteettomuus
  - testattavuus
  - selkeys
  - yksinkertaisuus

Laadulla on merkitystä

- Periaatteet juontavat 1970-luvun tutkimukseen
  - Parnas 1972: *information hiding*
  - Stevens, Myers & Constantine 1974: koheesio ja kytkentä (coupling)

Laadulla on merkitystä

- Periaatteet juontavat 1970-luvun tutkimukseen
  - Parnas 1972: *information hiding*
  - Stevens, Myers & Constantine 1974: koheesio ja kytkentä (coupling)
- Empiiristä näyttöä
  - Basili ym. 1996: luokat, joilla paljon riippuvuuksia, ovat virhealttiimpia
  - Tornhill & Borg 2022: 39 koodikantaa, heikkolaatuisessa koodissa 15 kertaa enemmän virheitä ja muutokset yli kaksi kertaa hitaampia

Suunnittelumallit

- *Suunnittelumalli* (design pattern): nimetty ratkaisu toistuvaan suunnitteluongelmaan
  - auttavat luomaan sisäisesti laadukasta koodia
  - ei valmista koodia vaan idea, joka sovitetaan tilanteeseen
  - nimet antavat yhteisen sanaston

Suunnittelumallit

- *Suunnittelumalli* (design pattern): nimetty ratkaisu toistuvaan suunnitteluongelmaan
  - auttavat luomaan sisäisesti laadukasta koodia
  - ei valmista koodia vaan idea, joka sovitetaan tilanteeseen
  - nimet antavat yhteisen sanaston
- Idea rakennusarkkitehtuurista (Christopher Alexander 1977)
- Tunnetuksi teki kirja *Design Patterns* (1994, *Gang of Four*)
  - 23 mallia, esim. tehdas, rakentaja, dekoraattori, strategia, komento

Suunnittelumallit

- *Suunnittelumalli* (design pattern): nimetty ratkaisu toistuvaan suunnitteluongelmaan
  - auttavat luomaan sisäisesti laadukasta koodia
  - ei valmista koodia vaan idea, joka sovitetaan tilanteeseen
  - nimet antavat yhteisen sanaston
- Idea rakennusarkkitehtuurista (Christopher Alexander 1977)
- Tunnetuksi teki kirja *Design Patterns* (1994, *Gang of Four*)
  - 23 mallia, esim. tehdas, rakentaja, dekoraattori, strategia, komento
- Kritiikkiä: moni malli paikkaa 1990-luvun oliokielten rajoitteita
  - innokas käyttö johtaa helposti tarpeettoman monimutkaiseen koodiin

Suunnittelumallit

- *Suunnittelumalli* (design pattern): nimetty ratkaisu toistuvaan suunnitteluongelmaan
  - auttavat luomaan sisäisesti laadukasta koodia
  - ei valmista koodia vaan idea, joka sovitetaan tilanteeseen
  - nimet antavat yhteisen sanaston
- Idea rakennusarkkitehtuurista (Christopher Alexander 1977)
- Tunnetuksi teki kirja *Design Patterns* (1994, *Gang of Four*)
  - 23 mallia, esim. tehdas, rakentaja, dekoraattori, strategia, komento
- Kritiikkiä: moni malli paikkaa 1990-luvun oliokielten rajoitteita
  - innokas käyttö johtaa helposti tarpeettoman monimutkaiseen koodiin
- Kurssilla nähty jo *dependency injection* ja *repository*, lisää viikkojen 6 ja 7 laskareissa

TAUKO 10 min

Koodin laatuattribuutti: kapselointi

- *Oliomuuttujat piilotetaan, tarvittaessa aksessorimetodit*

Koodin laatuattribuutti: kapselointi

- *Oliomuuttujat piilotetaan, tarvittaessa aksessorimetodit*

```
class Ostoskori:
    def __init__(self):
        self._ostokset = []

    def lisaa(self, tuote):
        self._ostokset.append(tuote)

    def hinta(self):
        return sum(t.hinta for t in self._ostokset)
```

Koodin laatuattribuutti: kapselointi

- *Oliomuuttujat piilotetaan, tarvittaessa aksessorimetodit*

```
class Ostoskori:
    def __init__(self):
        self._ostokset = []

    def lisaa(self, tuote):
        self._ostokset.append(tuote)

    def hinta(self):
        return sum(t.hinta for t in self._ostokset)
```

- Tallennustapa voidaan vaihtaa, esim. sanakirjaksi tuote $\rightarrow$ määrä, ilman että käyttävä koodi muuttuu

Kapselointi laajemmin

- Miksi kapseloidaan: muu koodi riippuu vain rajapinnasta, joten toteutusta voi muuttaa rikkomatta muuta koodia

Kapselointi laajemmin

- Miksi kapseloidaan: muu koodi riippuu vain rajapinnasta, joten toteutusta voi muuttaa rikkomatta muuta koodia
- Kapseloidaan mikä tahansa asia, joka voi muuttua
  - olion sisäinen tila
  - käytettävän olion tyyppi
  - käytetty algoritmi
  - olioiden luomisen tapa
  - käytettävän komponentin rakenne

Kapselointi laajemmin

- Miksi kapseloidaan: muu koodi riippuu vain rajapinnasta, joten toteutusta voi muuttaa rikkomatta muuta koodia
- Kapseloidaan mikä tahansa asia, joka voi muuttua
  - olion sisäinen tila
  - käytettävän olion tyyppi
  - käytetty algoritmi
  - olioiden luomisen tapa
  - käytettävän komponentin rakenne
- Myös arkkitehtuurin tasolla
  - kerrosarkkitehtuuri: ylempi kerros käyttää vain alemman kerroksen rajapintaa
  - mikropalvelu kapseloi logiikkansa ja tietonsa, tarjoaa vain verkkorajapinnan

Koodin laatuattribuutti: koheesio

Koodin laatuattribuutti: koheesio

- *Koheesio*:
  - kuinka pitkälle metodin, luokan tai komponentin koodi keskittyy tietyn yksittäisen toiminnallisuuden toteuttamiseen
  - hyvänä pidetään mahdollisimman korkeaa koheesion astetta

Koodin laatuattribuutti: koheesio

- *Koheesio*:
  - kuinka pitkälle metodin, luokan tai komponentin koodi keskittyy tietyn yksittäisen toiminnallisuuden toteuttamiseen
  - hyvänä pidetään mahdollisimman korkeaa koheesion astetta
- Luokkatason koheesio
  - luokan *vastuulla* vain yksi asia, tunnetaan myös nimellä *single responsibility principle*

Koheesio Flask-sovelluksessa

```python
@app.route("/create_todo", methods=["POST"])
def todo_creation():
    content = request.form.get("content")
    try:
        if len(content) < 5:
            raise UserInputError("Todo content length must be greater than 4")
        if len(content) > 100:
            raise UserInputError("Todo content length must be smaller than 100")

        sql = text("INSERT INTO todos (content) VALUES (:content)")
        db.session.execute(sql, {"content": content})
        db.session.commit()

        return redirect("/")
    except Exception as error:
        flash(str(error))
        return redirect("/new_todo")
```

Koheesio Flask-sovelluksessa

```python
@app.route("/create_todo", methods=["POST"])
def todo_creation():
    content = request.form.get("content")
    try:
        if len(content) < 5:
            raise UserInputError("Todo content length must be greater than 4")
        if len(content) > 100:
            raise UserInputError("Todo content length must be smaller than 100")

        sql = text("INSERT INTO todos (content) VALUES (:content)")
        db.session.execute(sql, {"content": content})
        db.session.commit()

        return redirect("/")
    except Exception as error:
        flash(str(error))
        return redirect("/new_todo")
```

- Huono koheesio: sama funktio käsittelee HTTP-pyynnön, validoi syötteen ja tekee tietokantaoperaatiot

Koheesio Flask-sovelluksessa

```python
@app.route("/create_todo", methods=["POST"])
def todo_creation():
    content = request.form.get("content")
    try:
        validate_todo(content)
        create_todo(content)
        return redirect("/")
    except Exception as error:
        flash(str(error))
        return redirect("/new_todo")

# util.py
def validate_todo(content):
    if len(content) < 5:
        raise UserInputError("Todo content length must be greater than 4")
    if len(content) > 100:
        raise UserInputError("Todo content length must be smaller than 100")

# todo_repository.py
def create_todo(content):
    sql = text("INSERT INTO todos (content) VALUES (:content)")
    db.session.execute(sql, {"content": content})
    db.session.commit()
```

Koheesio Flask-sovelluksessa

```python
@app.route("/create_todo", methods=["POST"])
def todo_creation():
    content = request.form.get("content")
    try:
        validate_todo(content)
        create_todo(content)
        return redirect("/")
    except Exception as error:
        flash(str(error))
        return redirect("/new_todo")

# util.py
def validate_todo(content):
    if len(content) < 5:
        raise UserInputError("Todo content length must be greater than 4")
    if len(content) > 100:
        raise UserInputError("Todo content length must be smaller than 100")

# todo_repository.py
def create_todo(content):
    sql = text("INSERT INTO todos (content) VALUES (:content)")
    db.session.execute(sql, {"content": content})
    db.session.commit()
```

- *delegoidaan osa vastuista* eri komponenteille

Miksi korkea koheesio?

- Muutos kohdistuu yhteen paikkaan
  - Robert Martin: luokalla vain *yksi syy muuttua*

Miksi korkea koheesio?

- Muutos kohdistuu yhteen paikkaan
  - Robert Martin: luokalla vain *yksi syy muuttua*
- Helpompi ymmärtää: nimi kertoo, mitä luokka tai funktio tekee

Miksi korkea koheesio?

- Muutos kohdistuu yhteen paikkaan
  - Robert Martin: luokalla vain *yksi syy muuttua*
- Helpompi ymmärtää: nimi kertoo, mitä luokka tai funktio tekee
- Helpompi testata: esim. validointia voi testata ilman HTTP-pyyntöä ja tietokantaa

Miksi korkea koheesio?

- Muutos kohdistuu yhteen paikkaan
  - Robert Martin: luokalla vain *yksi syy muuttua*
- Helpompi ymmärtää: nimi kertoo, mitä luokka tai funktio tekee
- Helpompi testata: esim. validointia voi testata ilman HTTP-pyyntöä ja tietokantaa
- Uudelleenkäytettävyys: samaa validointia voi käyttää myös muualla, esim. REST-rajapinnassa

Miksi korkea koheesio?

- Muutos kohdistuu yhteen paikkaan
  - Robert Martin: luokalla vain *yksi syy muuttua*
- Helpompi ymmärtää: nimi kertoo, mitä luokka tai funktio tekee
- Helpompi testata: esim. validointia voi testata ilman HTTP-pyyntöä ja tietokantaa
- Uudelleenkäytettävyys: samaa validointia voi käyttää myös muualla, esim. REST-rajapinnassa
- Arkkitehtuurin tasolla
  - kerrosarkkitehtuuri: kukin kerros keskittyy oman abstraktiotasonsa asioihin
  - mikropalvelu toteuttaa yhden liiketoiminnallisen kokonaisuuden, esim. suosittelun

Koodin laatuattribuutti: riippuvuuksien vähäisyys

Koodin laatuattribuutti: riippuvuuksien vähäisyys

- Pyrkimys korkeaan koheesioon johtaa ohjelmiin, joissa suuri määrä olioita/komponentteja

Koodin laatuattribuutti: riippuvuuksien vähäisyys

- Pyrkimys korkeaan koheesioon johtaa ohjelmiin, joissa suuri määrä olioita/komponentteja
- Olioiden oltava keskenään vuorovaikutuksessa toteuttaakseen ohjelman toiminnallisuuden: *paljon keskinäisiä riippuvuuksia*

Koodin laatuattribuutti: riippuvuuksien vähäisyys

- Pyrkimys korkeaan koheesioon johtaa ohjelmiin, joissa suuri määrä olioita/komponentteja
- Olioiden oltava keskenään vuorovaikutuksessa toteuttaakseen ohjelman toiminnallisuuden: *paljon keskinäisiä riippuvuuksia*
- *Riippuvuuksien vähäisyyden* periaate
  - eliminoidaan *tarpeettomat* riippuvuudet
  - sekä riippuvuudet konkreettisiin asioihin

Koodin laatuattribuutti: riippuvuuksien vähäisyys

- Pyrkimys korkeaan koheesioon johtaa ohjelmiin, joissa suuri määrä olioita/komponentteja
- Olioiden oltava keskenään vuorovaikutuksessa toteuttaakseen ohjelman toiminnallisuuden: *paljon keskinäisiä riippuvuuksia*
- *Riippuvuuksien vähäisyyden* periaate
  - eliminoidaan *tarpeettomat* riippuvuudet
  - sekä riippuvuudet konkreettisiin asioihin
- Hyödynnetään *dependency injection* -suunnittelumallia

Koodin laatuattribuutti: riippuvuuksien vähäisyys

```
def main():
    stats = StatisticsService()

class StatisticsService:
    def __init__(self):
        reader = PlayerReader()
```

vs

```
def main():
    reader = PlayerReader("https://studies.cs.helsinki.fi/nhlstats/2021-22/players.txt")
    stats = StatisticsService(reader)

class StatisticsService:
    def __init__(self, reader):
        self._players = reader.get_players()
```

Koodin laatuattribuutti: toisteettomuus

Koodin laatuattribuutti: toisteettomuus

- Aloittelevaa ohjelmoijaa pelotellaan toisteisuuden vaaroista uran ensiaskelista alkaen: älä copypastaa koodia!

Koodin laatuattribuutti: toisteettomuus

- Aloittelevaa ohjelmoijaa pelotellaan toisteisuuden vaaroista uran ensiaskelista alkaen: älä copypastaa koodia!
- Alan piireissä toisteisuudesta varoittava periaate kulkee nimellä DRY, don't repeat yourself
  - *every piece of knowledge must have a single, unambiguous, authoritative representation within a system*

Koodin laatuattribuutti: toisteettomuus

- Aloittelevaa ohjelmoijaa pelotellaan toisteisuuden vaaroista uran ensiaskelista alkaen: älä copypastaa koodia!
- Alan piireissä toisteisuudesta varoittava periaate kulkee nimellä DRY, don't repeat yourself
  - *every piece of knowledge must have a single, unambiguous, authoritative representation within a system*
- Koodin lisäksi periaate ulottuu koskemaan järjestelmän muitakin osia
  - tietokantaskeemaa, testejä, build-skriptejä

Koodin laatuattribuutti: toisteettomuus

- Aloittelevaa ohjelmoijaa pelotellaan toisteisuuden vaaroista uran ensiaskelista alkaen: älä copypastaa koodia!
- Alan piireissä toisteisuudesta varoittava periaate kulkee nimellä DRY, don't repeat yourself
  - *every piece of knowledge must have a single, unambiguous, authoritative representation within a system*
- Koodin lisäksi periaate ulottuu koskemaan järjestelmän muitakin osia
  - tietokantaskeemaa, testejä, build-skriptejä
- Suoraviivainen copypaste helppo eliminoida metodien avulla
  - kaikki toisteisuus ei ole yhtä ilmeistä, monissa suunnittelumalleissa kyse hienovaraisempien toisteisuuden muotojen eliminoinnista

Koodin laatuattribuutti: toisteettomuus

- Aloittelevaa ohjelmoijaa pelotellaan toisteisuuden vaaroista uran ensiaskelista alkaen: älä copypastaa koodia!
- Alan piireissä toisteisuudesta varoittava periaate kulkee nimellä DRY, don't repeat yourself
  - *every piece of knowledge must have a single, unambiguous, authoritative representation within a system*
- Koodin lisäksi periaate ulottuu koskemaan järjestelmän muitakin osia
  - tietokantaskeemaa, testejä, build-skriptejä
- Suoraviivainen copypaste helppo eliminoida metodien avulla
  - kaikki toisteisuus ei ole yhtä ilmeistä, monissa suunnittelumalleissa kyse hienovaraisempien toisteisuuden muotojen eliminoinnista
- Hyvä vs. paha copypaste
  - *three strikes and you refactor*

Koodin laatuattribuutti: testattavuus

Koodin laatuattribuutti: testattavuus

- Laadukas koodi on helppo testata kattavasti yksikkö- ja integraatiotestein
  - seuraa yleensä siitä, että koodi koostuu löyhästi kytketyistä, selkeän vastuun omaavista komponenteista

Koodin laatuattribuutti: testattavuus

- Laadukas koodi on helppo testata kattavasti yksikkö- ja integraatiotestein
  - seuraa yleensä siitä, että koodi koostuu löyhästi kytketyistä, selkeän vastuun omaavista komponenteista
- Hyvää testattavuutta auttaa turhien riippuvuuksien eliminointi dependency injection -periaatteen avulla

Koodin laatuattribuutti: testattavuus

- Laadukas koodi on helppo testata kattavasti yksikkö- ja integraatiotestein
  - seuraa yleensä siitä, että koodi koostuu löyhästi kytketyistä, selkeän vastuun omaavista komponenteista
- Hyvää testattavuutta auttaa turhien riippuvuuksien eliminointi dependency injection -periaatteen avulla
- Test driven development tuottaa yleensä hyvin testattavaa koodia

Koodin laatuattribuutti: selkeys ja luettavuus

- Aikoinaan rajallisten resurssien takia koodi kirjoitettiin tiiviiksi ja tehokkaaksi luettavuuden kustannuksella

Koodin laatuattribuutti: selkeys ja luettavuus

- Aikoinaan rajallisten resurssien takia koodi kirjoitettiin tiiviiksi ja tehokkaaksi luettavuuden kustannuksella
- Nykytrendi: tehdään koodia, joka nimeämisen sekä rakenteen kautta ilmaisee hyvin sen, mitä koodi tekee

Koodin laatuattribuutti: selkeys ja luettavuus

- Aikoinaan rajallisten resurssien takia koodi kirjoitettiin tiiviiksi ja tehokkaaksi luettavuuden kustannuksella
- Nykytrendi: tehdään koodia, joka nimeämisen sekä rakenteen kautta ilmaisee hyvin sen, mitä koodi tekee
- Miksi selkeä koodi on tärkeää?
  - Xia ym. 2018: 78 ammattilaista, yli 3000 työtuntia, noin 58 % ajasta kului koodin ymmärtämiseen
  - Robert Martin: lukemisen ja kirjoittamisen suhde jopa 10:1
  - oma aikoinaan niin selkeä koodi ei enää olekaan yhtä selkeää parin kuukauden kuluttua

Koodin laatuattribuutti: yksinkertaisuus

- Suunnittelumallit lisäävät koodin joustavuutta, mutta joustavuudella on hintansa
  - luokkien määrä kasvaa, toiminnan hahmottaminen vaikeutuu

Koodin laatuattribuutti: yksinkertaisuus

- Suunnittelumallit lisäävät koodin joustavuutta, mutta joustavuudella on hintansa
  - luokkien määrä kasvaa, toiminnan hahmottaminen vaikeutuu
- KISS: *keep it simple, stupid*
- YAGNI: *you aren't gonna need it* (Extreme Programming)
  - ei toiminnallisuutta tai joustavuutta sen varalta, että sitä ehkä joskus tarvitaan

Koodin laatuattribuutti: yksinkertaisuus

- Suunnittelumallit lisäävät koodin joustavuutta, mutta joustavuudella on hintansa
  - luokkien määrä kasvaa, toiminnan hahmottaminen vaikeutuu
- KISS: *keep it simple, stupid*
- YAGNI: *you aren't gonna need it* (Extreme Programming)
  - ei toiminnallisuutta tai joustavuutta sen varalta, että sitä ehkä joskus tarvitaan
- Suunnittelumalli käyttöön vasta todelliseen tarpeeseen
  - testattu ja siisti koodi on helppo refaktoroida joustavammaksi myöhemmin

Koodin laatuattribuutti: yksinkertaisuus

- Suunnittelumallit lisäävät koodin joustavuutta, mutta joustavuudella on hintansa
  - luokkien määrä kasvaa, toiminnan hahmottaminen vaikeutuu
- KISS: *keep it simple, stupid*
- YAGNI: *you aren't gonna need it* (Extreme Programming)
  - ei toiminnallisuutta tai joustavuutta sen varalta, että sitä ehkä joskus tarvitaan
- Suunnittelumalli käyttöön vasta todelliseen tarpeeseen
  - testattu ja siisti koodi on helppo refaktoroida joustavammaksi myöhemmin
- Ajankohtainen myös koodausagenttien kanssa: agentit tuottavat helposti tarpeettomia abstraktioita

Code smell

- Koodi ei ole aina hyvää...

Code smell

- Koodi ei ole aina hyvää...
- Martin Fowlerin mukaan
  - *koodihaju* (code smell) on helposti huomattava merkki siitä että koodissa on jotain pielessä
  - jopa aloitteleva ohjelmoija saattaa pystyä havaitsemaan koodihajun
  - sen takana oleva todellinen syy voi olla jossain syvemmällä

Code smell

- Koodi ei ole aina hyvää...
- Martin Fowlerin mukaan
  - *koodihaju* (code smell) on helposti huomattava merkki siitä että koodissa on jotain pielessä
  - jopa aloitteleva ohjelmoija saattaa pystyä havaitsemaan koodihajun
  - sen takana oleva todellinen syy voi olla jossain syvemmällä
- Koodihaju siis kertoo, että syystä tai toisesta *koodin sisäinen laatu* ei ole parhaalla mahdollisella tasolla

Code smell

- Koodi ei ole aina hyvää...
- Martin Fowlerin mukaan
  - *koodihaju* (code smell) on helposti huomattava merkki siitä että koodissa on jotain pielessä
  - jopa aloitteleva ohjelmoija saattaa pystyä havaitsemaan koodihajun
  - sen takana oleva todellinen syy voi olla jossain syvemmällä
- Koodihaju siis kertoo, että syystä tai toisesta *koodin sisäinen laatu* ei ole parhaalla mahdollisella tasolla
- Khomh ym. 2012: hajuisia luokkia muutetaan useammin ja niissä on useammin virheitä

Koodihajuja

- Koodihajuja on hyvin monenlaisia ja monentasoisia
- Esimerkkejä helposti tunnistettavista hajuista:
  - toisteinen koodi
  - liian pitkät metodit
  - luokat joissa on liikaa oliomuuttujia
  - luokat joissa on liikaa koodia
  - metodien liian pitkät parametrilistat
  - epäselkeät muuttujien, metodien tai luokkien nimet
  - kommentit, jotka selittävät *mitä* koodi tekee
    - *miksi*-kommentit ja docstringit ovat arvokkaita

Koodihajuja

- Koodihajuja on hyvin monenlaisia ja monentasoisia
- Esimerkkejä helposti tunnistettavista hajuista:
  - toisteinen koodi
  - liian pitkät metodit
  - luokat joissa on liikaa oliomuuttujia
  - luokat joissa on liikaa koodia
  - metodien liian pitkät parametrilistat
  - epäselkeät muuttujien, metodien tai luokkien nimet
  - kommentit, jotka selittävät *mitä* koodi tekee
    - *miksi*-kommentit ja docstringit ovat arvokkaita
- Pari monimutkaisempaa
  - Primitive obsession
  - Shotgun surgery

Refaktorointi

Refaktorointi

- Lääke koodin sisäisen laadun ongelmiin on *refaktorointi*
  - koodin toiminnallisuuden ennallaan pitävä sisäisen rakenteen muutos

Refaktorointi

- Lääke koodin sisäisen laadun ongelmiin on *refaktorointi*
  - koodin toiminnallisuuden ennallaan pitävä sisäisen rakenteen muutos
- Koodin rakennetta parantavia refaktorointeja on lukuisia, mm.
  - *rename variable/method/class*
  - *extract method*
  - *move field/method*
  - *extract superclass*

Refaktorointi

- Lääke koodin sisäisen laadun ongelmiin on *refaktorointi*
  - koodin toiminnallisuuden ennallaan pitävä sisäisen rakenteen muutos
- Koodin rakennetta parantavia refaktorointeja on lukuisia, mm.
  - *rename variable/method/class*
  - *extract method*
  - *move field/method*
  - *extract superclass*
- Osa pystytään tekemään sovelluskehitysympäristön avustamana
  - helpompaa staattisesti tyypitetyillä kielillä kuten Java

Miten refaktorointi kannattaa tehdä

- Refaktoroinnin edellytys on kattavien testien olemassaolo

Miten refaktorointi kannattaa tehdä

- Refaktoroinnin edellytys on kattavien testien olemassaolo
- Eteneminen pienin askelin
  - yksi hallittu muutos kerrallaan
  - testit suoritettava mahdollisimman usein

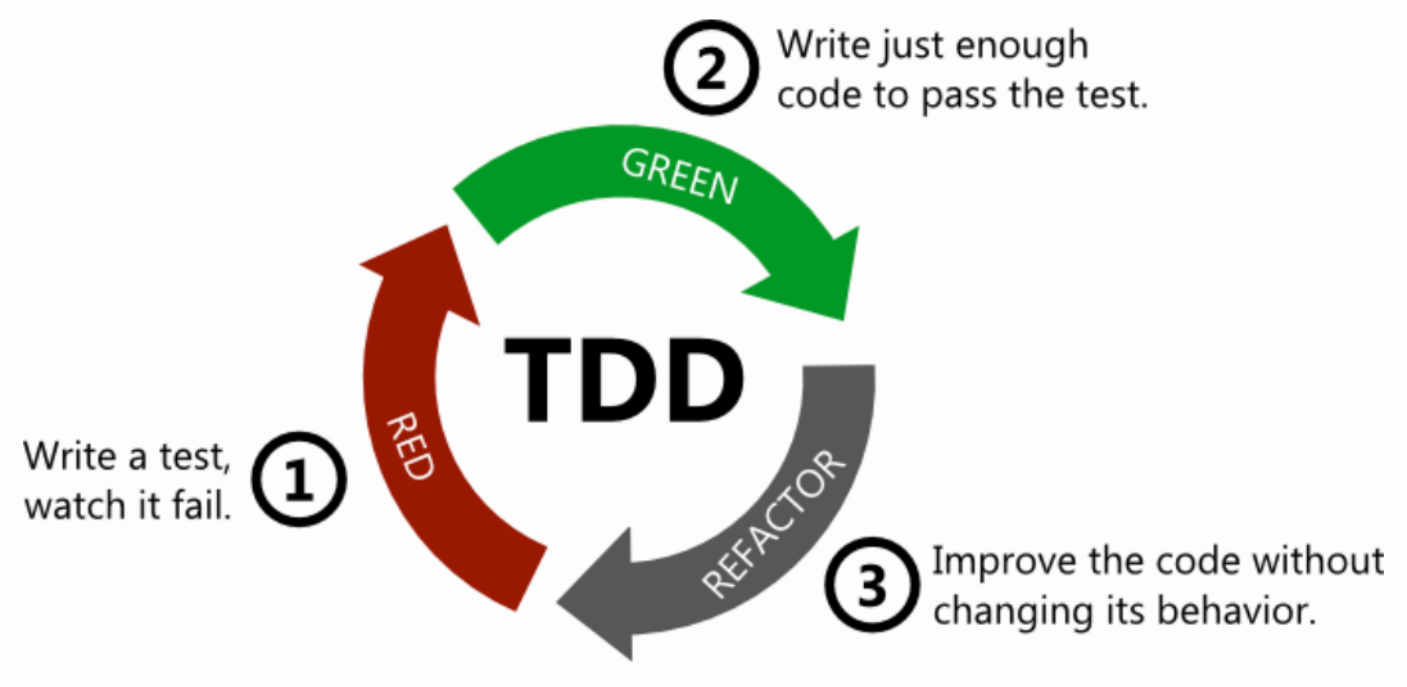
Miten refaktorointi kannattaa tehdä

- Refaktoroinnin edellytys on kattavien testien olemassaolo
- Eteneminen pienin askelin
  - yksi hallittu muutos kerrallaan
  - testit suoritettava mahdollisimman usein
- Refaktorointia kannattaa suorittaa lähes jatkuvasti
  - pitää koodin rakenteen selkeänä ja helpottaa sekä nopeuttaa koodin laajentamista

Miten refaktorointi kannattaa tehdä

- Refaktoroinnin edellytys on kattavien testien olemassaolo
- Eteneminen pienin askelin
  - yksi hallittu muutos kerrallaan
  - testit suoritettava mahdollisimman usein
- Refaktorointia kannattaa suorittaa lähes jatkuvasti
  - pitää koodin rakenteen selkeänä ja helpottaa sekä nopeuttaa koodin laajentamista
- Osa refaktoroinnista on helppoa ja suoraviivaista, aina ei näin
  - joskus tarve tehdä isoja, jopa viikkojen kestoisia refaktorointeja, joissa ohjelman rakenne muuttuu paljon

Refaktorointi tärkeä osa Test driven development -menetelmää



1. Kirjoitetaan sen verran testiä että testi ei mene läpi
2. Kirjoitetaan koodia sen verran, että testi menee läpi
3. **Jos huomataan koodin rakenteen menneen huonoksi refaktoroidaan koodin rakenne paremmaksi**
4. Jatketaan askeleesta 1

Refaktorointi ja koodausagentit

- Agentti optimoi helposti toimivuutta, ylläpidettävyyden kustannuksella: lisää uutta, harvoin siistii vanhaa
  - GitClear 2025: copypasten osuus muutetuista riveistä 8,3 % → 12,3 %, refaktorointiin liittyvän siirretyn koodin osuus 25 % → alle 10 %

Refaktorointi ja koodausagentit

- Agentti optimoi helposti toimivuutta, ylläpidettävyyden kustannuksella: lisää uutta, harvoin siistii vanhaa
  - GitClear 2025: copypasten osuus muutetuista riveistä 8,3 % → 12,3 %, refaktorointiin liittyvän siirretyn koodin osuus 25 % → alle 10 %
- Refaktorointi on pyydettävä erikseen
  - red-green-refactor myös agentin kanssa: ensin toimivaksi, sitten siistiksi
  - periaatteet ohjetiedostoon (AGENTS.md): *hyödynnä olemassa olevaa koodia, älä kopioi*

Refaktorointi ja koodausagentit

- Agentti optimoi helposti toimivuutta, ylläpidettävyyden kustannuksella: lisää uutta, harvoin siistii vanhaa
  - GitClear 2025: copypasten osuus muutetuista riveistä 8,3 % → 12,3 %, refaktorointiin liittyvän siirretyn koodin osuus 25 % → alle 10 %
- Refaktorointi on pyydettävä erikseen
  - red-green-refactor myös agentin kanssa: ensin toimivaksi, sitten siistiksi
  - periaatteet ohjetiedostoon (AGENTS.md): *hyödynnä olemassa olevaa koodia, älä kopioi*
- Toisaalta refaktoroinnista on tullut halpaa: agentti tekee sen nopeasti, kunhan testit ovat kunnossa

Tekninen velka

Tekninen velka

- Koodi ei ole aina laadultaan optimaalista

Tekninen velka

- Koodi ei ole aina laadultaan optimaalista
- Huonoa suunnittelua tai/ja ohjelmointia kuvaa käsite *tekninen velka* (technical debt)
  - termin otti käyttöön Ward Cunningham 1992

Tekninen velka

- Koodi ei ole aina laadultaan optimaalista
- Huonoa suunnittelua tai/ja ohjelmointia kuvaa käsite *tekninen velka* (technical debt)
  - termin otti käyttöön Ward Cunningham 1992
- Oikoteitä ottamalla saadaan ehkä nopeasti aikaan jotain
  - hätäinen ratkaisu tullaan maksamaan korkoineen takaisin *jos* ohjelmaa on tarkoitus laajentaa

Tekninen velka

- Koodi ei ole aina laadultaan optimaalista
- Huonoa suunnittelua tai/ja ohjelmointia kuvaa käsite *tekninen velka* (technical debt)
  - termin otti käyttöön Ward Cunningham 1992
- Oikoteitä ottamalla saadaan ehkä nopeasti aikaan jotain
  - hätäinen ratkaisu tullaan maksamaan korkoineen takaisin *jos* ohjelmaa on tarkoitus laajentaa
- Jos korkojen maksun aikaa ei koskaan tule, voi “huono koodi” olla asiakkaan etu
  - esim. minimum viable product (MVP)

Tekninen velka

- Koodi ei ole aina laadultaan optimaalista
- Huonoa suunnittelua tai/ja ohjelmointia kuvaa käsite *tekninen velka* (technical debt)
  - termin otti käyttöön Ward Cunningham 1992
- Oikoteitä ottamalla saadaan ehkä nopeasti aikaan jotain
  - hätäinen ratkaisu tullaan maksamaan korkoineen takaisin *jos* ohjelmaa on tarkoitus laajentaa
- Jos korkojen maksun aikaa ei koskaan tule, voi “huono koodi” olla asiakkaan etu
  - esim. minimum viable product (MVP)
- Tekninen velka voi olla järkevää tai jopa välttämätöntä
  - tuote saadaan nopeammin markkinoille, kun toteutuksessa oiotaan tietoisesti ja korjataan myöhemmin

- Kaikki tekninen velka ei samanlaista, taustalla voi olla
  - holtittomuus, osaamattomuus, tietämättömyys tai tarkoituksella tehty päätös

- Kaikki tekninen velka ei samanlaista, taustalla voi olla
  - holtittomuus, osaamattomuus, tietämättömyys tai tarkoituksella tehty päätös
- Martin Fowler jaottelee teknisen velan neljään eri luokkaan:
  - Holtiton ja tietoinen: “meillä ei ole aikaa suunnittelulle”
  - Holtiton ja tahaton: “mikä on kerrosarkkitehtuuri?”
  - Harkittu ja tahaton: “nyt tiedämme, miten se olisi pitänyt tehdä”
  - **Harkittu ja tietoinen: “julkaistaan nyt, seurauksista huolehditaan myöhemmin”**

- Kaikki tekninen velka ei samanlaista, taustalla voi olla
  - holtittomuus, osaamattomuus, tietämättömyys tai tarkoituksella tehty päätös
- Martin Fowler jaottelee teknisen velan neljään eri luokkaan:
  - Holtiton ja tietoinen: “meillä ei ole aikaa suunnittelulle”
  - Holtiton ja tahaton: “mikä on kerrosarkkitehtuuri?”
  - Harkittu ja tahaton: “nyt tiedämme, miten se olisi pitänyt tehdä”
  - **Harkittu ja tietoinen: “julkaistaan nyt, seurauksista huolehditaan myöhemmin”**
- Joskus tekninen velka pakottaa koodaamaan koko järjestelmän uudelleen

- Kaikki tekninen velka ei samanlaista, taustalla voi olla
  - holtittomuus, osaamattomuus, tietämättömyys tai tarkoituksella tehty päätös
- Martin Fowler jaottelee teknisen velan neljään eri luokkaan:
  - Holtiton ja tietoinen: “meillä ei ole aikaa suunnittelulle”
  - Holtiton ja tahaton: “mikä on kerrosarkkitehtuuri?”
  - Harkittu ja tahaton: “nyt tiedämme, miten se olisi pitänyt tehdä”
  - **Harkittu ja tietoinen: “julkaistaan nyt, seurauksista huolehditaan myöhemmin”**
- Joskus tekninen velka pakottaa koodaamaan koko järjestelmän uudelleen
- Velasta on puhuttava: käsitellään tiimin yhteisenä asiana, ei etsitä aiheuttajaa

Vibe-koodaus ja velka

- Vibe-koodaus: agentin koodi hyväksytään sitä lukematta, kunhan ohjelma näyttää toimivan

Vibe-koodaus ja velka

- Vibe-koodaus: agentin koodi hyväksytään sitä lukematta, kunhan ohjelma näyttää toimivan
- Seurauksena kahdenlaista velkaa
  - tekninen velka: copypastea, turhia abstraktioita, refaktoroimatonta koodia
  - kognitiivinen velka: tiimi ylläpitää koodia, jota kukaan ei ymmärrä

Vibe-koodaus ja velka

- Vibe-koodaus: agentin koodi hyväksytään sitä lukematta, kunhan ohjelma näyttää toimivan
- Seurauksena kahdenlaista velkaa
  - tekninen velka: copypastea, turhia abstraktioita, refaktoroimatonta koodia
  - kognitiivinen velka: tiimi ylläpitää koodia, jota kukaan ei ymmärrä
- Velka ei ole väistämätöntä
  - Borg ym. 2025, kontrolloitu koe: tekoälyn avulla tehtyä koodia ei ollut vaikeampi jatkokehittää
  - ratkaisevaa on agentin ohjaaminen ja tuotoksen katselmointi

Onko laatuattribuuteilla edelleen merkitystä?

- Kyllä, ainakin toistaiseksi

Onko laatuattribuuteilla edelleen merkitystä?

- Kyllä, ainakin toistaiseksi
- Laatu auttaa agenttia, joka näkee kerrallaan vain osan koodista
  - koheesio ja löyhä kytkentä pitävät muutoksen paikallisena
  - selkeät nimet auttavat löytämään muutoskohdan

Onko laatuattribuuteilla edelleen merkitystä?

- Kyllä, ainakin toistaiseksi
- Laatu auttaa agenttia, joka näkee kerrallaan vain osan koodista
  - koheesio ja löyhä kytkentä pitävät muutoksen paikallisena
  - selkeät nimet auttavat löytämään muutoskohdan
- Borg ym. 2026: kielimallit rikkoivat laadukasta koodia refaktoroidessaan 15–30 % harvemmin
  - ero hävisi järeimmillä malleilla

Onko laatuattribuuteilla edelleen merkitystä?

- Kyllä, ainakin toistaiseksi
- Laatu auttaa agenttia, joka näkee kerrallaan vain osan koodista
  - koheesio ja löyhä kytkentä pitävät muutoksen paikallisena
  - selkeät nimet auttavat löytämään muutoskohdan
- Borg ym. 2026: kielimallit rikkoivat laadukasta koodia refaktoroidessaan 15–30 % harvemmin
  - ero hävisi järeimmillä malleilla
- Laatu auttaa myös ihmistä: agentin tuotosta ei pysty katselmoimaan, jos koodi ei ole ymmärrettävää